Kurs Django — Lekcja 3

Widoki i szablony (templates)

Cel lekcji

Student potrafi renderować szablony HTML zamiast surowego HttpResponse, stosuje dziedziczenie szablonów i podłącza pliki statyczne (CSS).

Blok 1: Teoria (10–15 min)

Po co szablony, skoro HttpResponse działa?

HttpResponse ze sklejanym HTML-em w kodzie Pythona szybko staje się nieczytelny. Szablony oddzielają logikę (Python) od prezentacji (HTML) — to fundament wzorca MVT.

Dziedziczenie szablonów

Zamiast kopiować nagłówek/stopkę w każdym pliku HTML, tworzymy jeden base.html z wspólnym szkieletem, a pozostałe szablony go rozszerzają ({% extends %}) i nadpisują tylko konkretne fragmenty ({% block %}).

Blok 2: Ćwiczenia w terminalu

Krok 1 — konfiguracja folderu szablonów

Utworzyć strukturę:

```
mkdir -p templates/blog
```

W config/settings.py sprawdzić/ustawić DIRS:

```
TEMPLATES = [
    {
        ...
        'DIRS': [BASE_DIR / 'templates'],
        ...
    },
]
```

Krok 2 — szablon bazowy

templates/base.html:

```
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mój Blog{% endblock %}</title>
</head>
<body>
    <header><h1>Mój Blog</h1></header>
    <main>
```

```
        {% block content %}{% endblock %}
    </main>
    <footer><p>&copy; 2026 Mój Blog</p></footer>
</body>
</html>
```

Krok 3 — szablon dziedziczący

templates/blog/strona_glowna.html:

```
{% extends "base.html" %}

{% block title %}Strona główna{% endblock %}

{% block content %}
    <p>Witaj w moim blogu!</p>
{% endblock %}
```

Krok 4 — zmiana widoku na render()

W blog/views.py:

```
from django.shortcuts import render

def strona_glowna(request):
    return render(request, 'blog/strona_glowna.html')
```

Krok 5 — przekazywanie danych do szablonu (kontekst)

```
def strona_glowna(request):
    kontekst = {
        'tytul_powitania': 'Witaj na blogu!',
        'liczba_postow': 5,
    }
    return render(request, 'blog/strona_glowna.html', kontekst)
```

W szablonie:

```
{% block content %}
    <h2>{{ tytul_powitania }}</h2>
    <p>Liczba postów: {{ liczba_postow }}</p>
{% endblock %}
```

Krok 6 — pliki statyczne (CSS)

Struktura:

```
mkdir -p static/css
```

W config/settings.py:

```
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

static/css/style.css:

```
body { font-family: sans-serif; background: #f5f5f5; }
header { background: #333; color: white; padding: 1rem; }
```

W base.html (na górze pliku i w <head>):

```
{% load static %}
...
```

```
<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Krok 7 — test

```
python manage.py runserver
```

Sprawdzić czy CSS się ładuje (narzędzia deweloperskie przeglądarki → zakładka Network).

Blok 3: Zadanie dla studentów

- Stworzyć drugi szablon o_mnie.html dziedziczący po base.html
- Dodać do kontekstu listę (np. listę zainteresowań) i wyświetlić ją w szablonie przez {% for %}
- Dodać warunek {% if %} — np. wyświetlić inny komunikat, gdy liczba_postow wynosi 0

Checklist nawyków

☐ Nowy folder ze statykami/szablonami → zrestartować serwer deweloperski

☐ Każdy szablon dziedziczący zaczyna się od {% extends %}

☐ Pliki statyczne → pamiętać o {% load static %} na górze szablonu